

PROJECT PROPOSAL:

PROJECT PROPOSAL:

PharmaSync

Course/Section: BSIT 2B

Project Category: HealthTech

Date: April 7, 2026

PROJECT TEAM

Lead Integration Architect: Rhuwen C. Cantuba – *Responsible for API connectivity and system logic.*

Backend Developer: Josiah Luke Bustos– *Responsible for Database and Server-side scripting.*

Frontend Developer: Charles Kendrick V. Ubana– *Responsible for UI/UX and client-side validation.*

PROJECT DESCRIPTION

Current Situation:

In many municipalities like Basud, life-saving medicine often remains "invisible" to patients because each pharmacy operates on its own isolated, "siloed" computer system. This lack of communication forces patients to waste critical time visiting multiple stores only to find they are out of stock, while a third pharmacy nearby might actually have the medication they need.

The Proposed System:

The system solves the "invisible medicine" problem by acting as a digital bridge that connects all the different, separate pharmacy computers in Basud into one easy-to-use website. Its integrative design allows it to "talk" to many different types of software at once, gathering all their stock information into one central list, so medicine is no longer hidden in private files. The automation part means the website updates itself instantly whenever a bottle of medicine is sold, so the pharmacist doesn't have to type anything in manually. Instead of walking from store to store, a patient can simply check the site to see exactly which pharmacy has their life-saving medicine right now, saving them time and helping them get well faster.

SDG Alignment:

SDG 3: Good Health and Well-being. The core mission is improving healthcare delivery. Focuses on achieving universal health coverage, including access to safe, effective, quality, and affordable essential medicines and vaccines for all. It creates transparency by providing real-time availability, and it ensures that "access" isn't just about the medicine being in the town, but about the patient actually *finding* it when they need it.

TECHNICAL ARCHITECTURE

This section defines the "Integrative" nature of the project.

Table 1. Title

Component	Specification
Platform	Web-based Application (Responsive)
Frontend Stack	HTML5, CSS3, JavaScript
Backed Stack	PHP 8.x
Database Management	MYSQL (Relational Database)
Primary API Integration	PharmaSync Inventory Sync API
Data Exchange Format	JSON / RESTful Request
Integration Function	Our PHP code acts as a "Digital Translator." It receives data from different pharmacy systems and automatically updates our website's medicine list.

This table shows the tools we will use to build PharmaSync. We chose a web-based platform so that residents in Basud can easily check medicine availability on their phones or computers.

The most important part of our project is the API Integration. Since every pharmacy uses different software, our PHP code acts as a "Digital Bridge". It uses the JSON format to "talk" to these different systems, gathering their stock information into one central list. This automation means that whenever a pharmacy sells a medicine, our website updates itself instantly, saving patients from having to travel from store to store.

FUNCTIONAL REQUIREMENTS

A. User Module (For Patients/Residents)

- **Account Registration:** Users can create an account to save their frequent searches.
- **Medicine Search:** A simple search bar to find medicine names and prices.
- **Pharmacy Map:** Shows the location of the pharmacy that has the medicine.

B. Administrative Module

- **Inventory Dashboard:** A simple chart showing how much stock is left in the pharmacy.

- **Connection Logs:** A simple list showing if the "Inventory Sync" is working correctly.

UI/UX PROTOTYPE & DESIGN

Since you are including a UI prototype, it is helpful to visualize how the integration works behind the scenes.

Key Wireframes (Screenshots)

Place your mockup images here. For IPT projects, you must include the following four critical screens:



[Insert Screenshot of Login Page]

Figure 1. Login & Authentication (Sample Only)

The landing screen features a minimalist and intuitive entry point for users. Designed with a focus on accessibility, it utilizes a high-contrast blue-on-white typography for the PharmaSync brand identity. The interface provides two primary calls to action: a prominent Login button for returning users and a secondary Sign Up button for new registrations. This layout clearly defines the user's first steps within the application.

User Experience (UX) Flowchart

A diagram showing the simplified path of a resident: starting at the Landing Page → performing a Secure Login → using the Unified Search Engine to check for a specific medicine → and finally viewing the Real-Time Results showing which pharmacy in Basud has the item in stock.

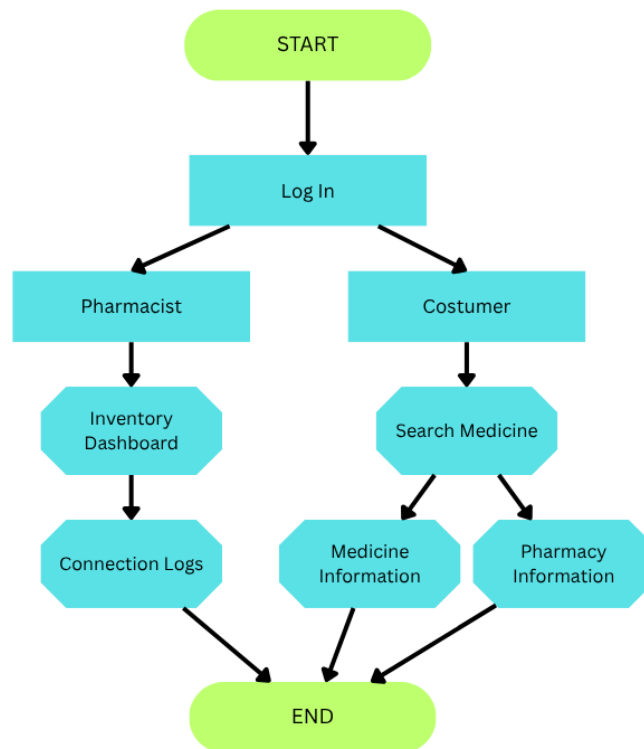


Figure 3. PharmaSync User Search and Availability Flowchart

SYSTEM DESIGN (DIAGRAMS)

Entity Relationship Diagram (ERD)

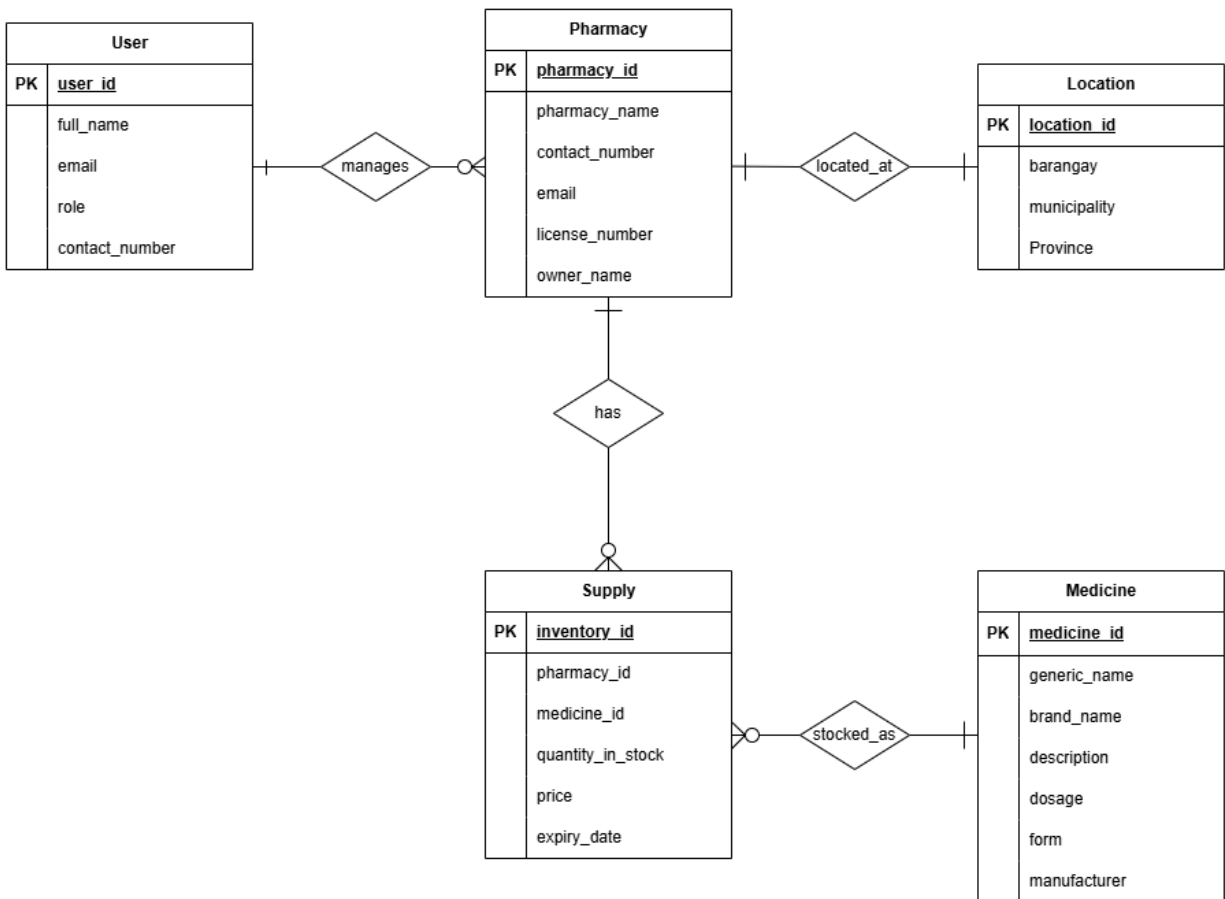


Figure 4. PharmaSync's Entity Relationship Diagram

Show how your UI inputs map to your database tables.

- **Table users:** handles login credentials.
- **Table transactions:** stores user requests captured via UI forms.
- **Table logs:** records API responses.

Use Case Diagram

(Showing how the User and Admin interact with the system).

Use Case Diagram: Single Patient Search (3 Pharmacies)

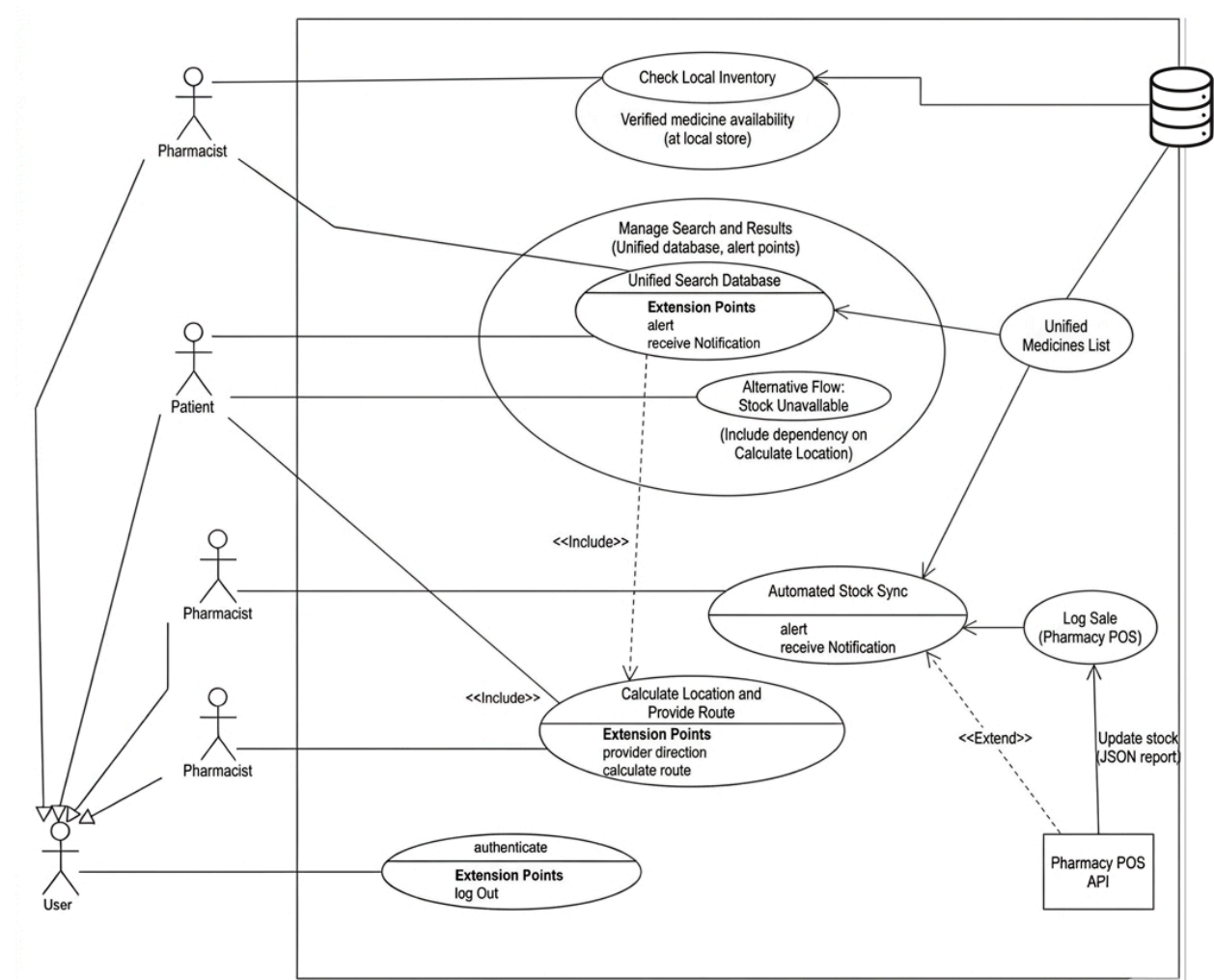


Figure 5. Medicine Search Use Cases

VI. DEVELOPMENT TIMELINE

Table 2. Title

Phase	Activities	Duration
Planning	Requirements gathering and ERD Design	1 Week
Development	Database setup, Frontend coding, Backend Logic	4 Week

Integration	API implementation and Middleware testing	1 Week
Deployment	UAT(User Acceptance Testing) and Final Demo	1 Week

1. Planning: This phase sets the foundation by identifying all necessary data points, such as Purok locations and medicine types, to ensure the system architecture is solid.
2. Development: Work focuses on crafting the responsive front-end and the core back-end functionality that allows the website to process pharmacy reports automatically.
3. Integration: This critical step involves linking the separate pharmacy computers to the central hub using the "Translator" logic we discussed earlier.
4. Deployment: The system is handed over to real users to verify that it accurately solves the "invisible medicine" problem and functions smoothly in a real-world setting.

Instructions for filling this out:

PRESENTATION REQUIREMENT (PPT)

IMPORTANT: All teams are required to create a PowerPoint Presentation for the proposal defense. The PPT must include:

1. Clear identification of the Lead Architect, Backend, and Frontend developers.
2. A demonstration of the UI flow using a tool like Figma or a sequence of screenshots.
3. A slide specifically explaining the PHP code or logic that handles the Primary API Integration.
4. Ensure the "Target Users" and "Features" match the entries previously submitted in the IPT Excel Project Registry.

Instructions for Submission:

1. Make sure your tables include primary keys (ID) and foreign keys to show how data links together.
2. Be very specific about which API you are using and how your PHP code handles the response.
3. Match the target user with what you have in your Excel file to ensure consistency.
4. Submit this proposal in PDF format and the accompanying PPT file before the scheduled defense.

Tab 2

To bring a project like the Basud "Digital Bridge" to life, you need integrations that handle real-time data syncing, secure communication, and user-facing notifications.

Given that pharmacies often use diverse, legacy Point of Sale (POS) systems, the primary integration focus should be on **Interoperability** and **Data Synchronization**.

1. Inventory & POS Integration (The Core)

Since the pharmacies use "separate computers," you need a way to pull data from their local databases (SQL, Excel, or proprietary software) into your central hub.

- **RESTful APIs:** The gold standard. You would build an API that each pharmacy's software "pings" whenever a transaction occurs.
- **Webhooks:** This is the "automation" part you mentioned. Instead of your site asking for updates, the pharmacy's system "pushes" the data to your site the moment a sale is made.
- **Inventory Management Systems (IMS):** Integrating with existing platforms like **Zoho Inventory** or **NetSuite** can simplify the process if the pharmacies already use them.

2. Data Standardisation (The "Language" Bridge)

Different pharmacies might name the same medicine differently (e.g., "Paracetamol 500mg" vs. "Acetaminophen 500mg").

- **FHIR (Fast Healthcare Interoperability Resources):** This is the international standard for exchanging healthcare information. Using FHIR ensures your "bridge" can talk to professional medical software without data errors.

3. Location & Mapping Services

To prevent patients from "walking store to store," the system needs to visualize where the medicine is.

- **Google Maps Platform / Mapbox API:** This allows you to show a real-time map of Basud with "pins" on pharmacies that currently have the stock in hand. It can also provide walking or driving directions.

4. Real-time Communication & Alerts

- **Twilio or WhatsApp Business API:** If a life-saving medicine is low on stock or has just been restocked, the system can automatically text a patient who has been looking for it.

- **Pusher or Socket.io:** These technologies enable the "instant update" feel on your website. When a bottle is sold, the number on the patient's screen changes without them needing to refresh the page.

Summary Table of Integrations

Feature	Primary Tool	Purpose
Data Sync	REST APIs / Webhooks	Connects pharmacy POS to the website.
Search	Algolia or Elasticsearch	Allows patients to find medicine names instantly.
Location	Google Maps API	Shows which store is closest to the patient.
Compliance	FHIR Standards	Ensures medical data is handled professionally.

Which of these pharmacy systems are you planning to connect first—is it mostly local custom software, or are they using standard retail POS systems? Think of this code as a **Digital Mailman**. Imagine a pharmacy in Basud wants to send your website a letter about their medicine stock. Here is how the Mailman handles it, line by line:

```
<?php
```

The "Start" Button.

This tells the computer: "Okay, stop reading regular text and start listening to my instructions!" It's like blowing a whistle to start a game.

```
$envelope = file_get_contents("php://input");
```

Catching the Letter.

- `$envelope`: This is a name we give to a "bucket" where we store the incoming message.
- `file_get_contents("php://input")`: This is the Mailman reaching out his hand to catch the letter coming through the internet. At this moment, the letter is still sealed, and the code hasn't read it yet.

```
$report = json_decode($envelope, true);
```

Opening and Translating.

- `json_decode`: The letter is written in a special computer language called JSON (which looks like messy code). This command is the "Translator." It opens the envelope and turns that messy code into a neat, organized list that PHP can actually read.
- `$report`: This is our new, clean list.

```
$pharmacy = $report['pharmacy_name'];
```

Reading the "Sender."

The code looks at the list and finds the part labeled "pharmacy_name." It copies that name (like "Pharmacy A") and saves it into a small box labeled `$pharmacy` so we don't forget it.

```
$medicine = $report['medicine'];
```

Reading the "Subject."

The code looks at the list again and finds the "medicine" label. It sees "Paracetamol" and saves that into another box labeled `$medicine`.

```
$stock = $report['current_stock'];
```

Reading the "Number."

Finally, it looks for the "current_stock" label. It sees the number **5** and saves it into a box labeled `$stock`. Now the Mailman knows exactly what is happening.

```
echo "System received a report...";
```

The "Shout-Out."

The `echo` command is like a megaphone. It takes all the info we saved in the boxes and "shouts" it onto the website screen so a human can see it: *"System received a report from Pharmacy A: Paracetamol is now at 5 units."*

```
?>
```

The "End" Sign.

This tells the computer: "I'm finished! You can take a break now."

Summary in a Nutshell

1. **Catch** the letter.
2. **Open** and translate it.
3. **Pick out** the important details.
4. **Announce** it to the world.